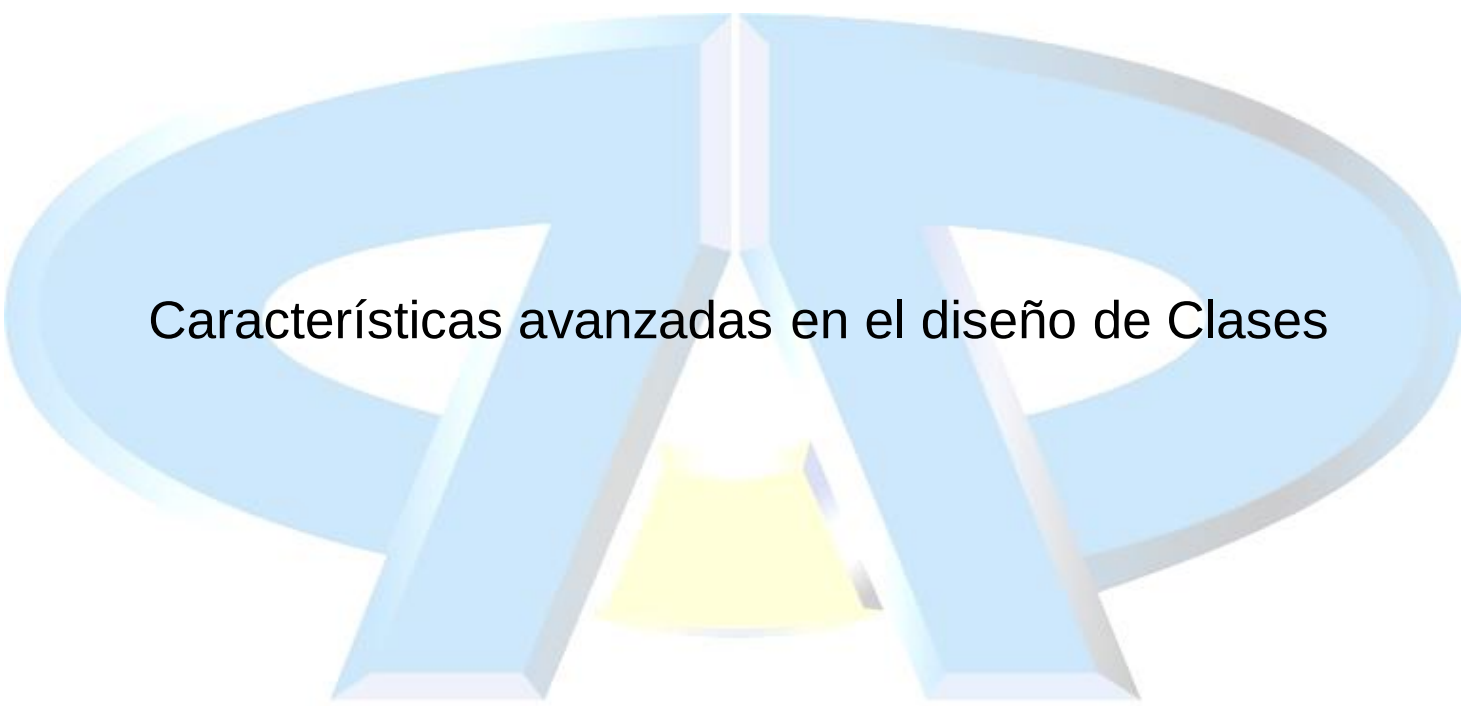




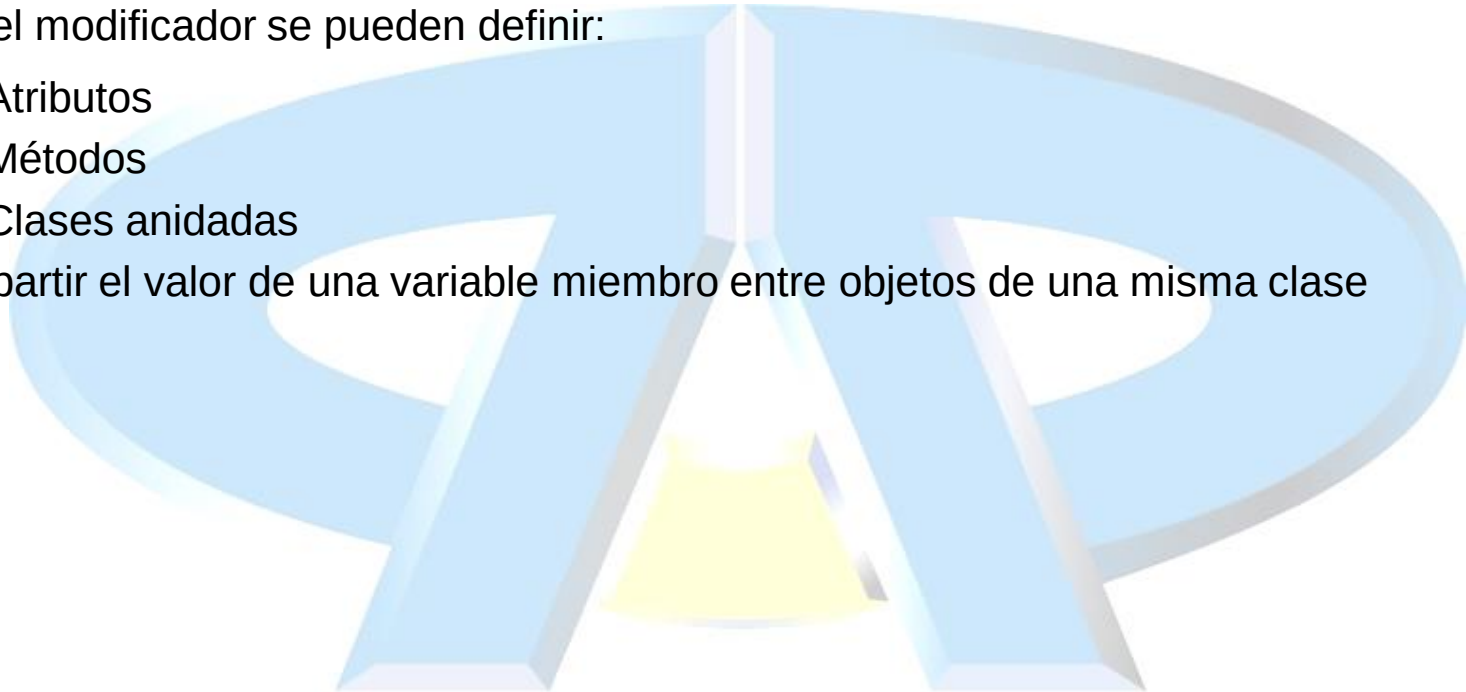
Características avanzadas en el diseño de Clases

Objetivos

- Empleo de variables estáticas, métodos y su inicialización
- Empleo de clases de utilidad. Patrón Singleton
- Uso y creación de modificar **final** en clases, métodos y variables
- Uso de tipos enumerados en Java
- Descripción del uso del modificador static en imports
- Descripción de clases abstractas y su proceso de creación. Métodos abstractos
- Descripción y uso de interfaces

Static

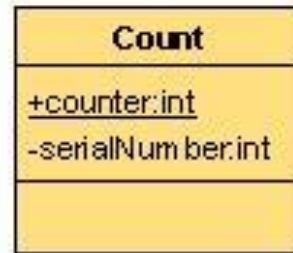
- Modificador de Java asociado con las clases más que con los objetos
- Con el modificador se pueden definir:
 - Atributos
 - Métodos
 - Clases anidadas
- Compartir el valor de una variable miembro entre objetos de una misma clase



Static – Atributos de clase

- Se quiere compartir una variable entre todas las instancias de una clase.
- Concepto de variable global en otros lenguajes.
- Ejemplo: uso de una variable de clase para la comunicación de las distintas instancias de una clase

```
public class Count {  
    private int serialNumber;  
    public static int counter;  
    public Count() {  
        counter++; //Incremento del contador  
        serialNumber = counter;  
    }  
}
```



- Cada objeto de la clase Count comparte la variable **counter** y cada vez que se crea un nuevo objeto Count la variable compartida **counter** se incrementa.

Static – Atributos de clase

- Con modificador publico se puede acceder a él desde fuera de la clase

```
Public class OtherClass {  
    public void incrementNumber(){  
        Count.counter++; //Acceso a la variable de clase  
    }  
}
```

- Todos los objetos Count comparten la misma zona de memoria para el atributos **counter**

Static – Métodos de clase

- Se necesita acceso a una variable estática de clase de clase pero no tiene el modificador public

```
public class Counter2 {  
    private static int counter=0;  
    private int serialNumber;  
    public Counter2(){  
        counter++;  
        serialNumber = counter;  
    }  
    public static int getCounter(){  
        return counter;  
    }  
    public static int getSerialNumber(){  
        return serialNumber; //Error de compilación  
    }  
    public int getSerialNumber(){  
        return serialNumber; //Acceso correcto  
    }  
}
```

Static – Métodos de clase

- Desde un método static sólo se puede acceder a variables

- static
- Variables locales del método
- Parámetros del método
- En otro caso Error del compilador

- Ejemplo de acceso a los métodos de clase:

```
public static void main(String[] args){  
    System.out.println("El número de objetos de la " +  
        "clase Counter2 es : " + Counter2.getCounter());  
    Counter2 count = new Counter2();  
    System.out.println("El número de objetos de la " +  
        "clase Counter2 es : " + Counter2.getCounter());  
}
```

- Métodos static no pueden ser sobrescritos. Sólo se pueden ocultar. En una Jerarquía de clases dos métodos iguales static no significa que exista sobrescritura, son dos métodos diferentes

Bloque de inicialización static

- Una clase puede tener uno o varios bloques de inicialización de tipo static. No forman parte de los métodos
- Los bloques static se ejecutan sólo una vez en la carga de la clase
- El orden de ejecución es según su aparición en el código fuente

```
public class StaticInit {  
    public static int count = 1;  
    static {  
        count = Integer.getInteger("myApplication.counter").intValue();  
    }  
}
```

```
public class TestStaticInit {  
    public static void main(String args[]) {  
        System.out.println("count = " + Count.counter);  
    }  
}
```

```
java -D myApplication.counter=47 TestStaticInit
```


Importación de elementos static

- Cuando se accede a un elemento static es necesario cualificar su referencia con el nombre de la clase de la cual procede
- Por ejemplo:
 - `double resultado = Math.cos(Math.PI * angulo);`
- Para evitar esta nomenclatura, se puede realizar la importación de los elementos static de dos formas:
 - Individualmente:
 - `import static java.lang.Math.PI;`
 - `import static java.lang.Math.cos;`
 - En grupo:
 - `import static java.lang.Math.*;`

Palabra reservada final

- Diferentes usos según el contexto en el que se emplea:
 - Clases
 - Métodos
 - Variables
- Clases final → La clase no puede ser extendida. Ejemplo: `java.lang.String` es declarada como final por razones de seguridad, se limita la introducción de código anómalo. Todos los métodos son final
- Métodos final → No pueden ser sobrescritos en subclases. Uso: implementaciones que no deben de ser cambiadas y que son cruciales para el estado consistente del objeto. Optimización por parte del compilador.
- Variable final → No pueden cambiar de valor. Cualquier intento de modificación provoca un Error de compilación.

```
Public class Bank{  
    Private static final double DEFAULT_INTEREST_RATE=3.2;//Constante  
}
```

Palabra reservada final

- Variables final son aquellas que sólo se declaran dejando su inicialización postergada. La inicialización de la variable debe de realizarse en el constructor. Una vez que tiene valor, este no puede ser modificado.

```
public class TestBlank {  
    private final int blankFinal;  
  
    public TestBlank() { } //Error de compilación  
  
    public int getBlankFinal() { return blankFinal; }  
    public void setBlankFinal(int x) { blankFinal = x; } //Error de Compilación  
  
    public static void main(String[] args) {  
        TestBlank t = new TestBlank();  
  
        System.out.println("t.blankFinal = " + t.getBlankFinal());  
        t.setBlankFinal(47); //Error de compilación  
        System.out.println("t.blankFinal = " + t.getBlankFinal());  
    }  
}
```

Tipos enumerados

- una variable tiene un único valor dentro de un conjunto de valores predefinidos. Ejemplo: Los días de la semana.
- Antes de j2se 1.5 los tipos enumerados se construían con constantes

```
public class DaysOfWeek {  
    public static final int MONDAY=0;  
    public static final int TUESDAY=1;  
    public static final int WEDNESDAY=2;  
    public static final int THURSDAY=3;  
    public static final int FRIDAY=4;  
    public static final int SATURDAY=5;  
    public static final int SUNDAY=6;  
  
    private int dayOfWeek;  
  
    public DaysOfWeek(int day){  
        this.dayOfWeek = day;  
    }  
}
```

Tipos enumerados

```
public int getDayOfWeek(){  
    return dayOfWeek;  
}  
}
```

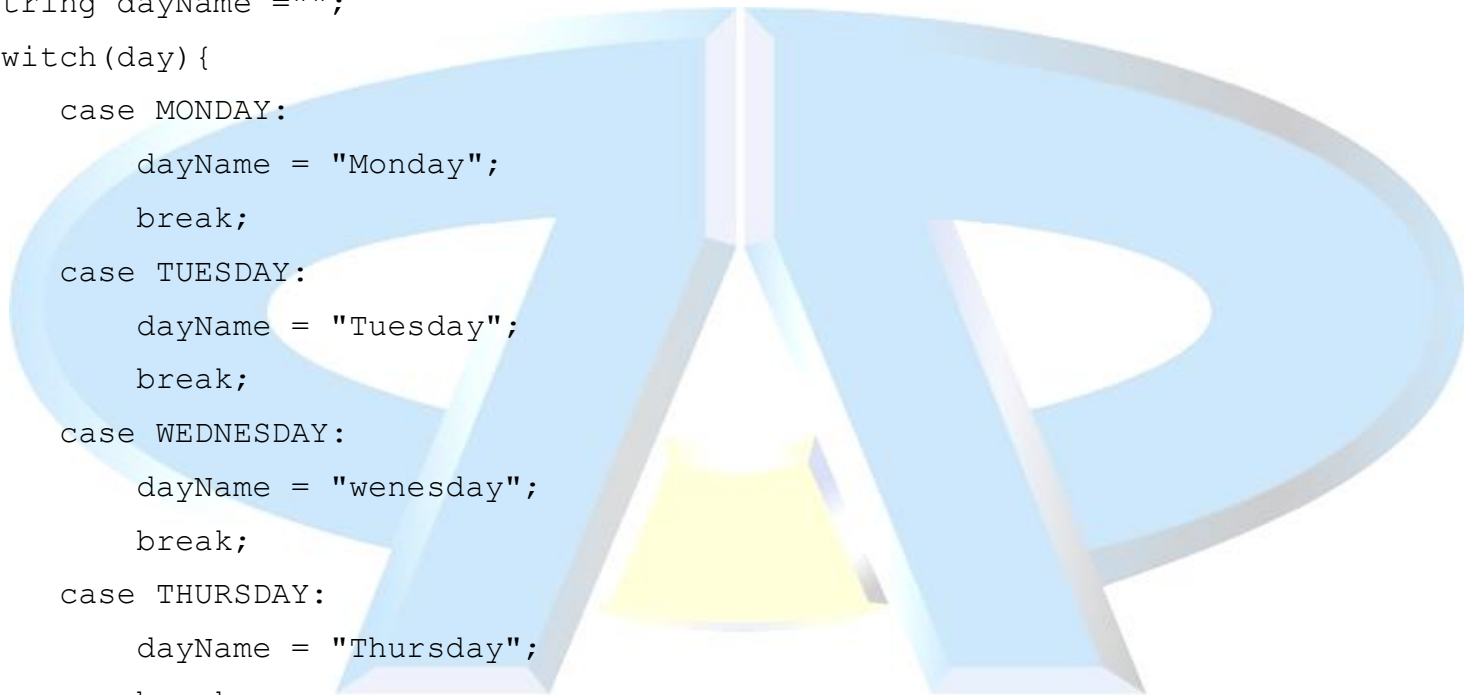
- **Código de prueba**

```
public static void main(String[] args){  
    DaysOfWeek days = new DaysOfWeek(DaysOfWeek.MONDAY);  
    System.out.println("Today is " + days.getDayOfWeek());  
}
```

- Se desea que el resultado mostrado no sea un número sino un día de la semana. Es necesario hacer un método para mostrar el día en texto

Tipos enumerados

```
public String getNameOfDay(int day){  
    String dayName = "";  
    switch(day){  
        case MONDAY:  
            dayName = "Monday";  
            break;  
        case TUESDAY:  
            dayName = "Tuesday";  
            break;  
        case WEDNESDAY:  
            dayName = "wenesday";  
            break;  
        case THURSDAY:  
            dayName = "Thursday";  
            break;  
        case SATURDAY:  
            dayName = "Saturday";  
            break;  
    }  
}
```



Tipos enumerados

```
case SUNDAY:
    dayName = "Sunday";
    break;
default:
    System.err.println("Invalid day of week");
}
return dayName;
}
```

- Código de prueba

```
public static void main(String[] args){
    DaysOfWeek days = new DaysOfWeek(DaysOfWeek.MONDAY);
    System.out.println("Today is " +
        days.getNameOfDay(days.getDayOfWeek()));
    DaysOfWeek days = new DaysOfWeek(35); //No existe
}
```

- Principales inconvenientes:

- Tipo no seguro. Los días de la semana se declaran como enteros, se puede pasar al constructor un entero mayor que 6 que no tendría sentido

Tipos enumerados

- No hay espacio de nombres. Se debe de proporcionar un prefijo para las constantes declaradas como días de la semana para que haya conflictos con otras tipos de datos declarados
- Valores poco significativos. Es necesario implementar un pretty printer para imprimir los días de la semana con su nombre
- Nuevo tipo de dato Enumerado definido a partir de j2se 1.5. El tipo enum
- La declaración de un tipo enum define una clase que puede incluir métodos y otros campos.
- Declaración:

```
public enum EnumDaysOfWeek{  
    MONDAY,  
    TUESDAY,  
    WEDNESDAY,  
    THURSDAY,  
    FRIDAY,  
    SATURDAY,  
    SUNDAY  
}
```


Tipos enumerados

- Con el uso de tipos de datos enumerados la clase DaysOfWeek se modifica:

```
public class DaysOfWeek {  
  
    private EnumDaysOfWeek day;  
  
    public DaysOfWeek(EnumDaysOfWeek day) {  
        this.day = day;  
    }  
    public EnumDaysOfWeek getDayOfWeek() {  
        return day;  
    }  
    public String getNameOfDay() {  
        String dayName = "";  
        switch(day) {  
            case MONDAY:  
                dayName = "Monday";  
                break;  
            case TUESDAY:  
                dayName = "Tuesday";  
                break;  
        }  
    }  
}
```

Tipos enumerados

```
case WEDNESDAY:
    dayName = "wenesday";
    break;
case THURSDAY:
    dayName = "Thursday";
    break;
case FRIDAY:
    dayName = "Friday";
    break;
case SATURDAY:
    dayName = "Saturday";
    break;
case SUNDAY:
    dayName = "Sunday";
    break;
}
return dayName;
}
```



Tipos enumerados

```
public static void main(String[] args){  
    DaysOfWeek days =  
        new DaysOfWeek(EnumDaysOfWeek.FRIDAY);  
    System.out.println("Today is " + days.getDayOfWeek());  
    System.out.println("Today is " +  
        days.getNameOfDay());  
    DaysOfWeek days2=  
        new DaysOfWeek(23); //Error de compilación  
}  
}
```

- Se desea el manejo de la descripción de los tipos enumerados al tipo de dato enum. Se definen constructores y métodos de la forma:

Tipos enumerados

```
public enum EnumDaysOfWeek{
    MONDAY ("Monday"),
    TUESDAY ("Tuesday"),
    WEDNESDAY ("Wednesday"),
    THURSDAY ("thursday"),
    FRIDAY ("Friday"),
    SATURDAY ("Saturday"),
    SUNDAY ("Sunday");

    private final String name;

    private EnumDaysOfWeek(String name){
        this.name = name;
    }

    public String getName(){
        return this.name;
    }
}
```

Tipos enumerados

- El constructor del tipo enumerado debería de ser empleado con el modificador private
- Los argumentos al constructor del tipo enumerado se proporcionan después de la declaración de cada uno de los valores
- Los tipos enumerados pueden tener un numero indeterminado de atributos y métodos
- Código de prueba final

```
public static void main(String[] args){  
    DaysOfWeek2 days =  
        new DaysOfWeek2(EnumDaysOfWeek.FRIDAY);  
    System.out.println("Today is " +  
        days.getDayOfWeek().getName());  
}
```

Clases Abstractas

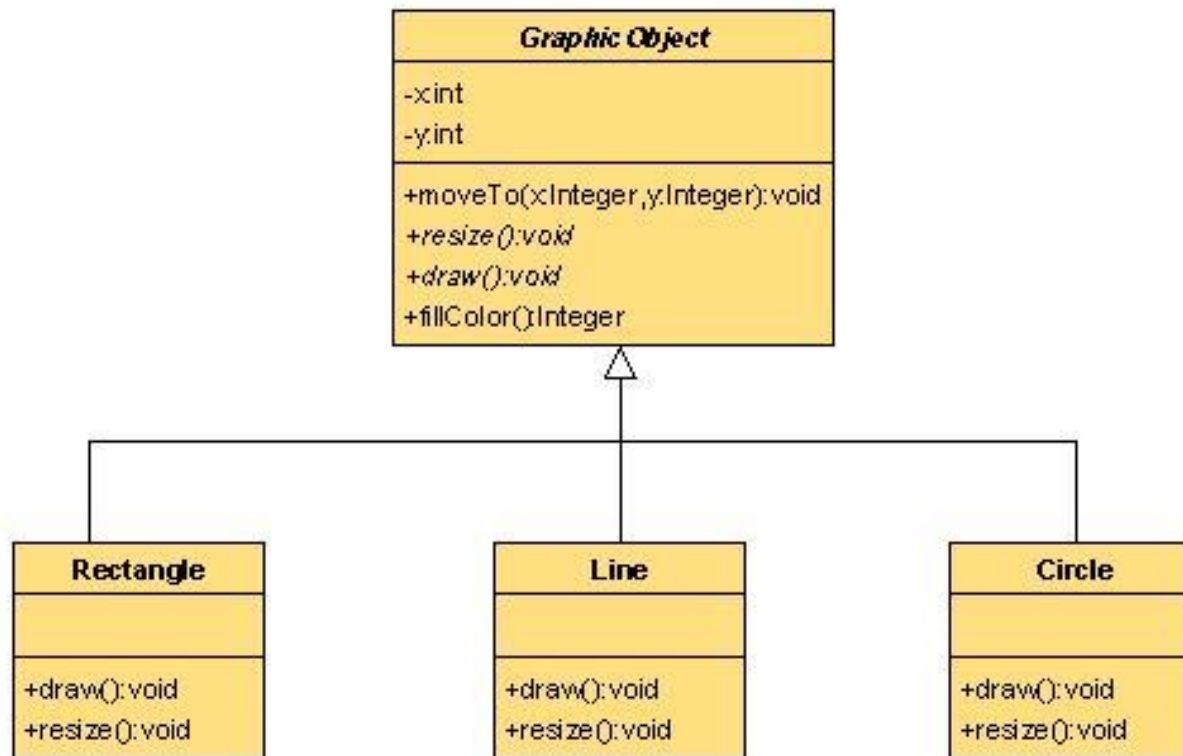
- **Motivación:** En una aplicación de dibujo orientada a objetos, se pueden dibujar círculos, rectángulos, líneas y muchos otros objetos gráficos. Todos estos objetos tienen en común ciertos estados:

- Posición
 - Orientación
 - Color de las líneas
 - Color de relleno
- y ciertos comportamientos.
- Mover a
 - Rotar
 - Redimensionarse
 - Dibujar

Algunos de estos estados y comportamientos son los mismos para todos los objetos (mover a, posición, color de relleno), mientras que otros requieren diferentes implementaciones según sea el objeto (redimensionar o dibujar)

Todos los objetos gráficos deben de conocer cómo dibujarse o redimensionarse a sí mismos, sólo difieren en cómo lo hacen.

Clases Abstractas



Clases Abstractas

- **Solución:** Declaración de una clase abstracta `GraphicObject` que proporciona variables y métodos que serán implementados y se comparten por todas las subclases como las variables de posición `x,y` y el método `moveTo`.
- Se debe de declarar en la clase abstracta los métodos cuya implementación recae en cada una de las clases debido a que requieren implementaciones diferentes (`redraw` o `resize`). Estos métodos serán abstractos.
- El código de la clase abstracta `GraphicObject` es de la forma:

```
abstract class GraphicObject {  
    int x, y;  
    ...  
    void moveTo(int newX, int newY) { ... }  
    abstract void draw();  
    abstract void resize();  
}
```


Clases Abstractas

- El código de las subclases concretas es de la forma:

```
class Circle extends GraphicObject {  
    void draw() { ... }  
    void resize() { ... }  
}  
  
class Rectangle extends GraphicObject {  
    void draw() { ... }  
    void resize() { ... }  
}
```

Clases Abstractas

- Una clase abstracta es una clase que es declarada como abstracta. Puede contener o no métodos abstractos
- No se pueden instanciar, pero pueden ser especializadas permitiendo herencia
- Un método abstracto se declara sin implementación (sin llaves) como:

```
abstract void moveTo(double deltaX, double deltaY);
```
- Si una clase contiene métodos abstractos debe de ser declarada como abstracta. Si una subclase de una clase abstracta no implementa todos los métodos abstractos → clase abstracta
- Pueden tener atributos, métodos concretos y constructores.
- Buena práctica → constructores protected debido a que sólo serán invocados por las subclases.

Interfaces

- **Motivación:** situación en la ingeniería del software en dónde distintos grupos de desarrollo software deben de estar de acuerdo en cómo su software interactúa. Cada grupo debe de ser capaz de escribir su código sin tener conocimiento de cómo otros grupos lo hacen.
- Sociedad futura en donde los coches son conducidos por robots y transportan a pasajeros sin la presencia de un operador humano
- Los fabricantes de automóviles escriben software (cualquier lenguaje orientación a objetos) que interactúa con el automóvil – parar, frenar, acelerar, girar izda., girar dcha., etc
- Otro sector industrial, por ejemplo fabricantes de instrumentos electrónicos de guiado, fabrican sistemas que por medio de la recepción inalámbrica de la posición del vehículo por GPS y el estado del tráfico son capaces de guiar el vehículo
- Los fabricantes de la industria del automóvil deben publicar un interfaz estándar para la industria que indique los métodos que deben de ser implementados para mover el vehículo (para cualquier vehículo de la industria del auto). Los fabricantes de sistema de guiado puede escribir su propio software implementando los métodos descritos en las interfaces publicadas. Ningún fabricante necesita conocer como otros fabricantes implementan su software, de hecho, cada fabricante considera su software propietario y se reserva los derechos de modificación

Interfaces

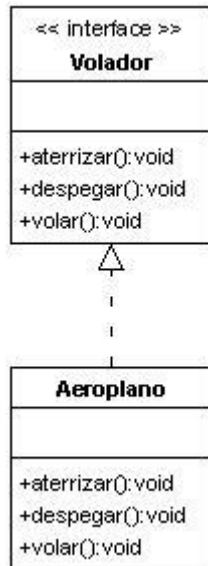
- Representa un contrato en dónde se define qué se debe de implementar pero no el cómo
- Todo método es abstracto y público sin necesidad de declararlo. Un interface en Java no implementa ninguno de los métodos que declara
- Todos los atributos que puede declarar son constantes (public static final)
- Una clase concreta que implementa una interfaz debe de definir e implementar todos los métodos declarados en la interfaz. Una clase puede implementar más de una interfaz (herencia múltiple)
- En una jerarquía de objetos donde las clases concretas pueden extender e implementar interfaces la identidad de los objetos se conoce mediante el uso del operador “instanceof”
- Declaración de una interfaz: la palabra clave interface, una lista de interfaces padre separadas por coma si hay, y el cuerpo de la interface. Ejemplo:

```
public interface GroupedInterface extends Interface1, Interface2, Interface3 {  
    // constant declarations  
    double E = 2.718282; // base of natural logarithms  
  
    // method signatures void doSomething (int i, double x);  
    int doSomethingElse(String s); }  

```

Interfaces

- Grupo de objetos con la misma capacidad: volar. Construcción de un interfaz que soporta las operaciones del grupo de objetos

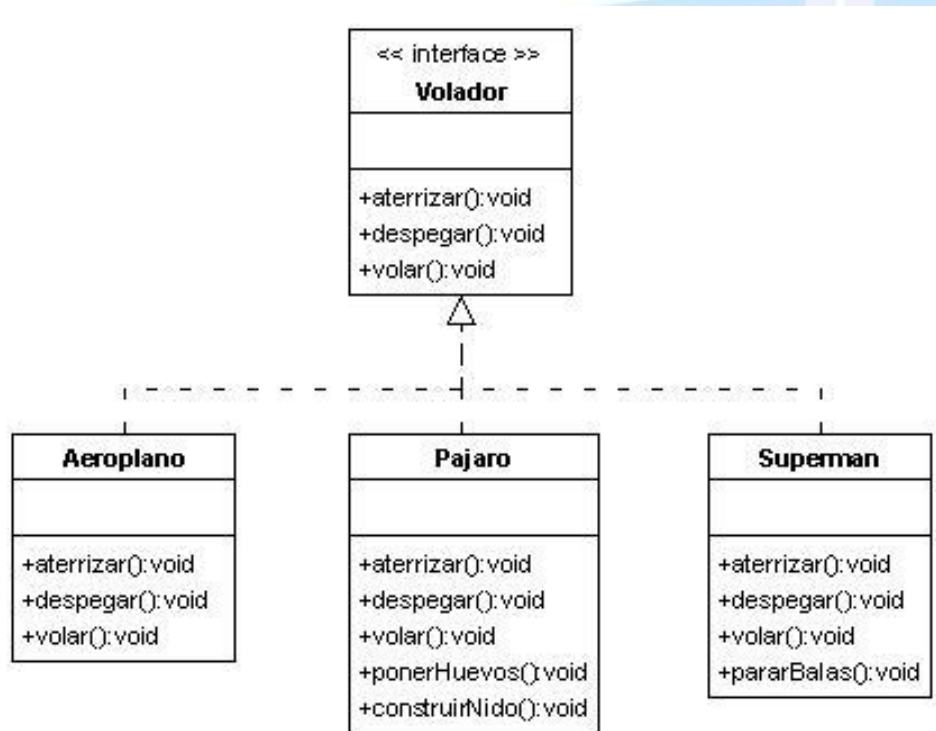


```
public interface Volador{
    void aterrizar(); //Es público aunque no lleve public
    public void despegar();
    public void volar();
}

public class Aeroplano implements Volador{
    public void aterrizar(){
        //Implementacion del metodo para Aeroplano
    }
    public void despegar(){
        //Implementación del método para Aeroplano
    }
    Public void volar(){
        //Implementación del metodo para Aeroplano
    }
}
```

Interfaces

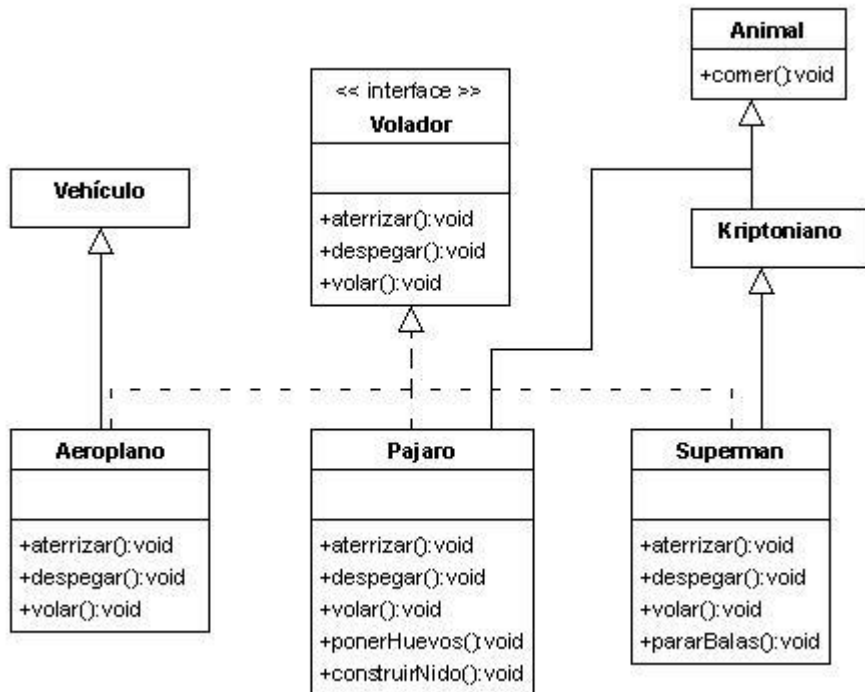
- Puede haber múltiples clases implementando la interfaz. Ejemplo:



- Un aeroplano es un tipo de vehículo que puede volar
- Un Pájaro de un tipo de animal y puede volar
- Superman es un tipo de animal originario de Krypton y puede volar

Interfaces – Herencia múltiple

- Problema herencia múltiple → Herencia de distintas implementaciones del mismo método
- En Java no se puede extender de más de una clase. Implementación de herencia múltiple mediante extensión de clases e implementación de interfaces



```
Public class Pajaro extends Animal implements
Volador{
    public void aterrizar(){//implementación}
    public void despegar(){//implementación}
    public void volar(){//implementación}
    public void ponerhuevos(){//Compartamiento}
    public void construirNido(){//Comport.}
    public void comer(){//Sobreescritura}
}
```